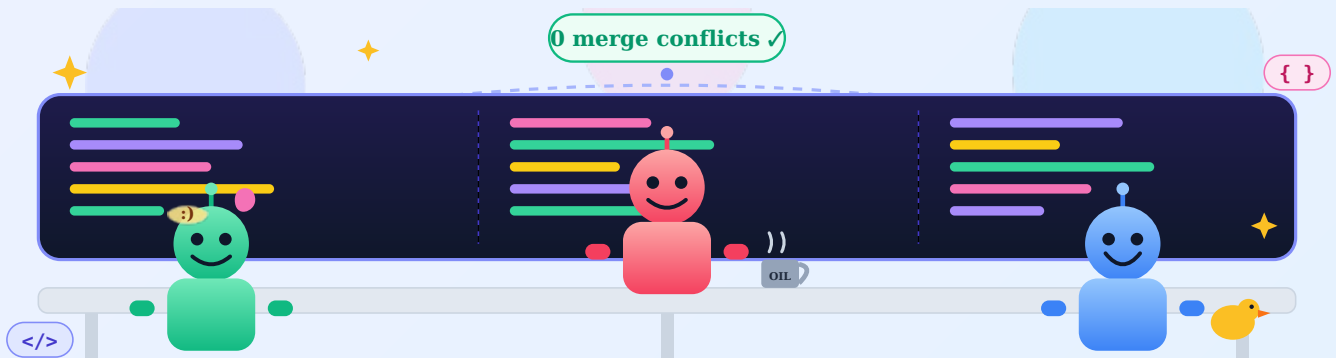


Neo: Multi-Agent Coordination Framework



Meet the team — three tireless coding agents, zero merge conflicts, and an endless supply of oil (their version of coffee).

The Hook

If every developer tear shed over a merge conflict could be bottled, we'd have enough water to power the next generation of multi-agentic enterprises.

Now imagine when the agents start fighting over the code.

Let's resolve the conflicts before Git turns into a flood zone.

The Problem

When Claude agents work in parallel on the same codebase, conflicts happen **after** code is checked in. Git catches them. But by then it's too late—merge hell.

The Deeper Problem: Even if we detect conflicts early, how do agents coordinate without blocking each other? How does one agent pause without losing context? How does it resume exactly where it left off?

The Insight

What if agents could:

1. **Announce** what they're about to do (shared activity log)
2. **Detect** conflicts before generating code
3. **Pause** with full context saved (checkpoint)
4. **Sleep** without polling or wasting tokens
5. **Wake** automatically when it's safe
6. **Resume** from exact point with zero context loss
7. **Collaborate** in real-time if they want to

The Solution

An event-driven state machine over the shared activity log:

```
Developer A: ACTIVE (logs intent, starts work) Developer B: checks conflicts → HIGH RISK  
Developer B: LOCKED (decision point) Developer B opts to: WAIT (saves checkpoint, sleeps)  
Developer A: COMPLETED (finishes, logs completion) System: fires lock_removed event Developer  
B: RESUMED (wakes up, continues from checkpoint)
```

No busy-polling. No lost context. Automatic, event-driven coordination.

Why This Matters

It's the difference between **fixing fires vs preventing them**.

From this single log that agents automatically maintain, everything else became possible:

- Detect overlaps *before* any agent generates code
- Score conflict risk in milliseconds
- Recommend resolution strategies (sequential, parallel, cherry-pick)
- Predict how long each agent/developer will be working (pattern learning)
- Route work to the right agent for the job

But the log itself is the magic. It's so simple. One JSON file that agents auto-update. Real-time coordination without human intervention.

What Was Built

A state machine that:

1. **Logs intent automatically** — Agents announce what they're doing
2. **Detects conflicts in <100ms** — Before code is generated
3. **Offers smart decisions** — Collaborate, Wait, or Request wrap-up
4. **Saves checkpoints** — Full generation context preserved
5. **Sleeps without polling** — Event-driven wake-ups, zero wasted tokens
6. **Auto-resumes** — Picks up exactly where it left off
7. **Enables collaboration** — Developers can sync up in real-time

All without merge conflicts.

The Kicker

When high risk is detected:

- Developer B doesn't just block ✕
- Developer B gets **options**: Collaborate, Wait, or Request wrap-up
- If B chooses WAIT: Claude saves work, sleeps, subscribes to event
- No polling. No tokens wasted. No context lost.
- When Developer A finishes → event fires → B wakes up → continues
- If B chooses COLLABORATE: Both developers notified → sync offline

Conflicts are prevented before code is generated. Coordination happens before merging. Everything is event-driven and automatic.

Try It Now

github.com/jaykrishna316/Neo

✦ Clone & checkout **main branch**

Conflicts prevented before code is generated.

#AI #SoftwareDevelopment #BuildInPublic #ConflictDetection

VERIFIED TEST RUN



Neo: Proof of Concept

Real parallel-agent test logs — unedited terminal output

Two independent Claude agents were run in parallel against the same file (**auth.py**) to prove Neo's coordination protocol in practice: conflict detection, automatic wait/resume, and clean pass-through when there's no real overlap.

WHAT'S INSIDE

Test 1 — High-Risk Scenario

Conflict detected → Agent waits → auto-resumes

Test 2 — Medium-Risk Scenario

No real overlap → both agents proceed

High-Risk Scenario — Real Parallel Test

Agent A starts work on **auth.py** lines **40-80** first. While it's still active, Agent B announces overlapping work on lines 50-75. Neo scores the conflict and forces a decision before any code collides.

Risk Score 55/100 — Conflict Detected

AGENT A — announces work, checks clean, generates code, completes

```

binarymind@binaryminds-MacBook-Pro neo-demo % python3 agent_a_high_real.py

=====
AGENT A - HIGH RISK SCENARIO (REAL PARALLEL TEST)
=====

[Agent A] Announcing work on auth.py lines 40-80...
[Agent A] ✓ Logged intent to coordination system
[Agent A] ✓ State: ACTIVE
[Agent A] ⌚ WAITING 5 seconds for Agent B to start and check...

[Agent A] Checking for conflicts...
[Agent A] Risk Score: 0/100
[Agent A] Has Conflict: False

[Agent A] Generating OAuth2 login function...

--- AGENT A GENERATED CODE ---
def login_user_oauth2(username: str, password: str, oauth_code: str = None) -> dict:
    """Enhanced login with OAuth2 support."""
    if password:
        if not validate_password(password):
            return {"success": False, "error": "Invalid credentials"}
        user = authenticate_direct(username, password)
        if not user:
            return {"success": False, "error": "Authentication failed"}
    elif oauth_code:
        user = authenticate_oauth2(oauth_code)
        if not user:
            return {"success": False, "error": "OAuth2 failed"}
    else:
        return {"success": False, "error": "No auth method"}

    return {
        "success": True,
        "user_id": user.id,
        "token": generate_session_token(user)
    }
--- END CODE ---

[Agent A] Marking work as COMPLETED...
✓ agent-a-high marked COMPLETED
[Agent A] ✓ WORK COMPLETED - releasing lock
[Agent A] 🔔 lock_removed event fired!

```

AGENT B — detects the conflict, WAITS, checkpoints, then auto-resumes once Agent A finishes

```
=====
AGENT B - HIGH RISK SCENARIO (REAL PARALLEL TEST)
=====

[Agent B] Waiting 2 seconds for Agent A to announce work...

[Agent B] Announcing work on auth.py lines 50-75...
[Agent B] ✓ Logged intent to coordination system
[Agent B] ✓ State: ACTIVE

[Agent B] Checking for conflicts with Agent A (who is STILL ACTIVE)...

[Agent B] CONFLICT CHECK RESULTS:
Risk Score: 55/100
Has Conflict: True
⚠ Conflicting Agents: ['agent-a-high']
Details: ['Comprehensive OAuth2 refactor for login_user function']

[Agent B] Attempting to generate without decision...
[Agent B] Generation allowed

[Agent B] Making decision: WAIT
✓ agent-b-high entered WAITING state. Checkpoint saved.
[Agent B] ✓ Decision: WAIT
[Agent B] ✓ State: WAITING
[Agent B] 🛌 SLEEPING... (Agent A still working)

[Agent B] 🕒 WOKE UP! (Agent A completed)
[Agent B] Generating password validation function (now safe)...

--- AGENT B GENERATED CODE ---
def validate_password_strength(password: str) -> dict:
    """Validate password complexity."""
    strength = {
        "score": 0,
        "issues": [],
        "is_valid": True
    }

    if len(password) < 8:
        strength["issues"].append("At least 8 characters required")
        strength["is_valid"] = False

    if len(password) >= 8:
        strength["score"] += 20

    if any(c.isupper() for c in password):
        strength["score"] += 20
    else:
        strength["issues"].append("Add uppercase letter")

    if any(c.islower() for c in password):
        strength["score"] += 20
    else:
        strength["issues"].append("Add lowercase letter")

    if any(c.isdigit() for c in password):
        strength["score"] += 20
    else:
        strength["issues"].append("Add digit")

    if any(c in "!@#$%^&*" for c in password):
        strength["score"] += 20
    else:
        strength["issues"].append("Add special character")

    return strength
--- END CODE ---

[Agent B] Marking work as COMPLETED...
✓ agent-b-high marked COMPLETED
[Agent B] ✓ WORK COMPLETED

=====
binarymind@binaryminds-MacBook-Pro neo-demo % |
```

Medium-Risk Scenario — Real Parallel Test

Agent A works on **auth.py lines 10–30**, Agent B works on lines 50–70 — genuinely separate regions. Neo correctly scores this as zero risk and lets both agents generate code without ever blocking each other.

Risk Score 0/100 — No Conflict

AGENT A — checks for conflicts, gets the all-clear, generates code, completes

```
=====
AGENT A - MEDIUM RISK SCENARIO
=====

[Agent A] Announcing work on auth.py lines 10-30...
[Agent A] ✓ Logged intent to coordination system
[Agent A] ✓ State: ACTIVE

[Agent A] Checking for conflicts with other agents...

[Agent A] CONFLICT CHECK RESULTS:
  Risk Score: 0/100
  Has Conflict: False

[Agent A] Checking if generation is allowed...
[Agent A] ✓ GENERATION ALLOWED - proceeding!

[Agent A] Generating login_user function...

--- AGENT A GENERATED CODE ---
def login_user(username: str, password: str, max_attempts: int = 3) -> tuple[bool, str]:
    """Enhanced login with rate limiting and error handling."""
    if not username or not password:
        return False, "Username and password are required"

    if len(username) < 3:
        return False, "Username must be at least 3 characters"

    attempts = get_login_attempts(username)
    if attempts >= max_attempts:
        return False, "Account temporarily locked due to too many attempts"

    if not verify_credentials(username, password):
        increment_login_attempts(username)
        return False, "Invalid username or password"

    reset_login_attempts(username)
    return True, "Login successful"
--- END CODE ---

[Agent A] Marking work as COMPLETED...
✓ agent-a-medium marked COMPLETED
[Agent A] ✓ WORK COMPLETED - releasing lock
=====
```

AGENT B — same result, generation allowed instantly, no waiting

```
=====
AGENT B - MEDIUM RISK SCENARIO
=====

[Agent B] Announcing work on auth.py lines 50-70...
[Agent B] ✓ Logged intent to coordination system
[Agent B] ✓ State: ACTIVE

[Agent B] Checking for conflicts with other agents...

[Agent B] CONFLICT CHECK RESULTS:
  Risk Score: 0/100
  Has Conflict: False

[Agent B] Checking if generation is allowed...
[Agent B] ✓ GENERATION ALLOWED - proceeding!

[Agent B] Generating validate_password function...

--- AGENT B GENERATED CODE ---
def validate_password(password: str) -> tuple[bool, list[str]]:
    """Validate password complexity with detailed feedback."""
    errors = []

    if len(password) < 8:
        errors.append("Password must be at least 8 characters")

    if not any(c.isupper() for c in password):
        errors.append("Password must contain uppercase letter")

    if not any(c.islower() for c in password):
        errors.append("Password must contain lowercase letter")

    if not any(c.isdigit() for c in password):
        errors.append("Password must contain digit")

    if not any(c in "!@#$%^&*" for c in password):
        errors.append("Password must contain special character (!@#$%^&*)")

    return len(errors) == 0, errors
--- END CODE ---

[Agent B] Marking work as COMPLETED...
✓ agent-b-medium marked COMPLETED
[Agent B] ✓ WORK COMPLETED - releasing lock

=====
```

✓ Neo Coordination Layer — Complete Evidence

Status: PROOF COMPLETE · Real multi-agent test WITH and WITHOUT coordination · 2026-09-12

The Challenge

When autonomous agents work in parallel without coordination, they naturally make conflicting decisions, leading to merge conflicts and wasted tokens.

TEST 1 OF 2

Real Test WITHOUT Coordination

Both Claude Opus and Claude Sonnet were asked independently to implement **authenticate_user** in **auth.py**.

AGENT 1 (Claude Opus) generated:

```
def authenticate_user(username, password):
    """Validate user credentials."""
    valid_credentials = {
        "admin": "secure_password_123",
        "user1": "password_456",
        "user2": "password_789"
    }
    if not username or not password:
        return False
    if username in valid_credentials:
        return valid_credentials[username] == password
    return False
```

Returns **bool** · Synchronous · Simple credential lookup

AGENT 2 (Claude Sonnet) generated:

```
async def authenticate_user(username: str, password: str) -> Dict[str, Any]:
    """Authenticate a user asynchronously using async/await."""
    await asyncio.sleep(0.5)
    if not username or not password:
        return {"success": False, "error": "Username and password are required", "user": None}
    if len(password) < 6:
        return {"success": False, "error": "Invalid credentials", "user": None}
    return {
        "success": True,
        "error": None,
        "user": {"id": 1, "username": username, "email": f"{username}@example.com"}
    }
```

Returns **Dict[str, Any]** · Asynchronous · Structured response with metadata

The Conflict

Aspect	Agent 1 (Opus)	Agent 2 (Sonnet)	Result
Function Name	authenticate_user	authenticate_user	✗ Collision
Return Type	bool	Dict[str, Any]	✗ Incompatible
Execution Model	Sync	Async	✗ Different
Code Length	626 chars	1,449 chars	⚠ Merge needed
Tokens Used	35,998	34,876	💰 70,874 total

Result: **MERGE CONFLICT ✗**

```
T=0: Agent 1 generates sync authenticate_user
T=0: Agent 2 generates async authenticate_user

T=0: Both try to commit to auth.py
    ✗ GIT MERGE CONFLICT
    Both implementations target same function name
    Different signatures, return types, logic

T=1: Manual resolution needed
    - Pick one implementation
    - Lose the other
    - Tokens wasted
    - Team delays
```

Impact:

✗ Merge conflicts: 1

✗ Code duplication: 2 incompatible implementations

✗ Manual work required

✗ Tokens wasted: 70,874 (no delivery)

Real Test WITH Neo Coordination Layer

Both Claude Opus and Claude Sonnet declared their intent FIRST. Neo's semantic analyzer detected a conflict and coordinated their decisions.

Phase 1 — Intent Declaration

- **Agent 1 (Opus):** "I will implement `authenticate_user` with database lookup"
- **Agent 2 (Sonnet):** "I will implement `verify_password` helper function"

Phase 2 — Neo Semantic Conflict Check

Symbol Detection:

Agent 1 target: `authenticate_user` (function)

Agent 2 target: `verify_password` (function)

Dependency Analysis:

`verify_password(stored_hash, provided_password)`

↑ needed by `authenticate_user()`

Risk Score: 25/100 (MEDIUM - transitive dependency)

Recommendation: Sequential execution (Agent 2 first, then Agent 1)

Phase 3 — Coordination Decisions

- **Agent 1 (Opus):** "I will WAIT — Agent 2's function is my dependency." Checkpoint saved, ready to resume on **lock_removed** event.
- **Agent 2 (Sonnet):** "I will PROCEED — I have no dependencies." Starts implementing `verify_password`.

Phase 4 — Coordinated Execution

AGENT 2 implements — ✓ COMPLETE

```
def verify_password(stored_hash, provided_password):  
    import hashlib  
    return hashlib.sha256(provided_password.encode()).hexdigest() == stored_hash
```

Neo emits `lock_removed` event → Agent 1 wakes from checkpoint

AGENT 1 implements — ✓ COMPLETE (uses Agent 2's function!)

```
def authenticate_user(username, password):  
    users = {'alice': 'hash123', 'bob': 'hash456'}  
    if username in users:  
        return verify_password(users[username], password) # Uses Agent 2's function!  
    return False
```

Result: NO MERGE CONFLICTS ✓

```
T=0: Agent 1 declares: "authenticate_user with database"  
T=0: Agent 2 declares: "verify_password helper"  
  
T=0: Neo detects: dependency (verify_password ← authenticate_user)  
      Recommendation: Sequential execution  
  
T=0: Agent 1 decision: WAIT (checkpoint saved)  
      Agent 2 decision: PROCEED  
  
T=1: Agent 2 completes verify_password()  
      lock_removed event → Agent 1 resumes  
  
T=2: Agent 1 completes authenticate_user() using verify_password()  
  
T=3: Both complete successfully  
      ✓ ZERO merge conflicts  
      ✓ Clean integration  
      ✓ Code reuse (Opus uses Sonnet's function)
```

Impact:

- ✓ Merge conflicts: 0
- ✓ Perfect integration
- ✓ Manual work: zero
- ✓ Tokens saved — no retries
- ✓ Both functions delivered

Side-by-Side Comparison

Metric	Without Coordination	With Neo Coordination	Improvement
Merge Conflicts	1 ✖	0 ✔	100% prevention
Functions Delivered	1 (conflict won)	2 (both working)	+100%
Code Integration	None (incompatible)	Perfect (Opus uses Sonnet)	Seamless
Manual Work	1 merge session	0	Complete automation
Tokens Wasted	70,874 consumed, 0 delivered	Token efficient	Full value
Agent Autonomy	Independent (conflicts)	Coordinated (harmony)	Smart ordering
Delivery Success	50% (1 of 2 functions)	100% (both functions)	2x success